TITLE

Air Theremin

TOPIC

Build Guide: from a blinking LED to playing music with your hands

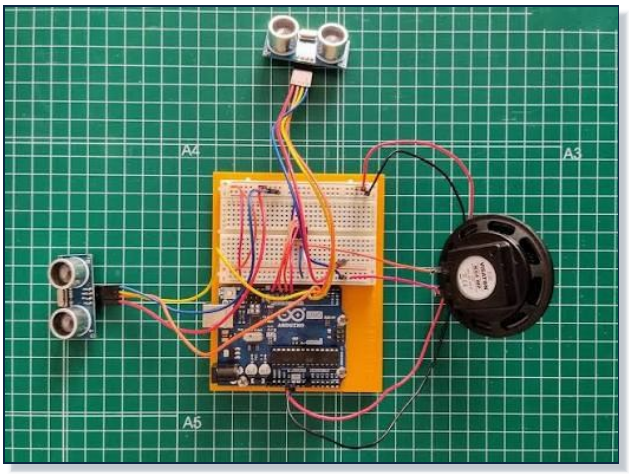
DEPARTMENT

IEEE SBUB · Project Resurrection

FULL NAME

Youcef HARMOUNI

The mission



Reference build: two ultrasonic sensors and a speaker on an Arduino Uno.

Play music without touching anything.

Right hand → pitch
Left hand → volume

DIFFICULTY

★★★☆☆

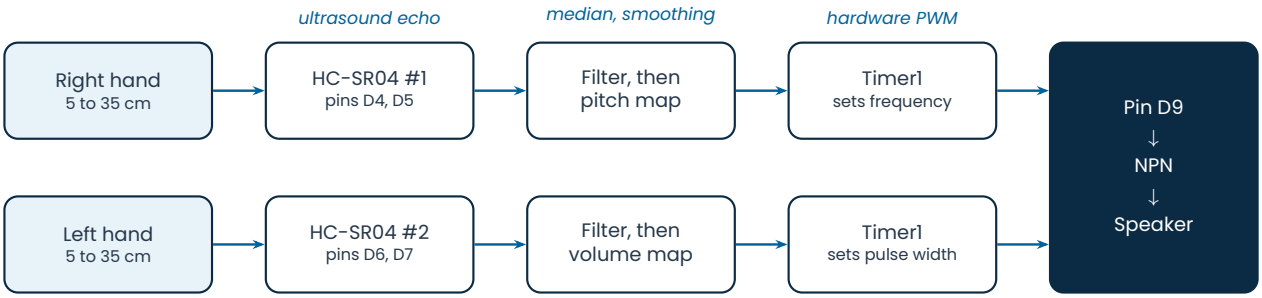
PITCH RANGE

220 Hz ↔ 880 Hz

YOU WILL LEARN

LED, button, LDR, PWM, echo ranging, timers, filtering

The whole machine on one picture



What you need

FOR THE THEREMIN

- 1 Arduino Uno + USB cable
- 2 HC-SR04 ultrasonic sensors
- 1 Breadboard and jumper wires
- 1 NPN transistor (2N3904 or PN2222)
- 1 1 kΩ resistor
- 1 100 Ω resistor
- 1 8 Ω small speaker
- 1 1N4148 diode

FOR THE PRACTICE STAGES

- 2 LEDs (red and green)
- 2 330 Ω resistors
- 1 10 kΩ resistor
- 1 Push button (4 legs)
- 1 LDR (photoresistor)
- 1 10 kΩ resistor for the LDR divider
- 1 Multimeter

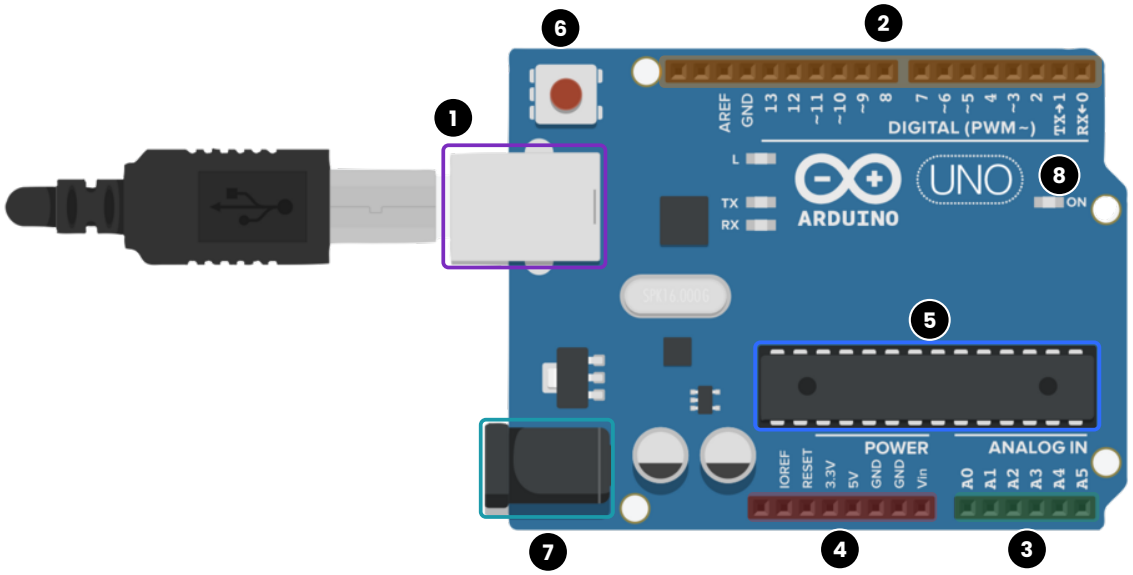
Your road map

1	2	3	4	5	6	7
Toolbox	**Blink**	**Button**	**LDR**	**Ultrasonic**	**Sound**	**Build**
upload code	output pins, timing	inputs, pull-ups	ADC, dividers, PWM	distance by echo	pitch, timers, duty	assemble, test

1 Your toolbox

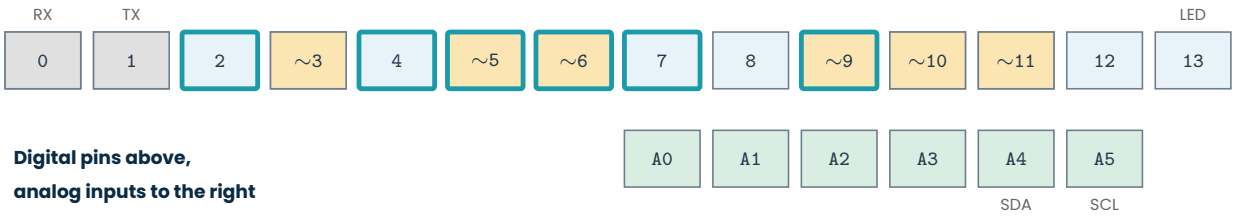
The Arduino, the IDE, a little electricity, and the breadboard

1.1 Meet the Arduino Uno



1. **USB**: power, upload code, talk to the PC
2. **Digital pins** 0 to 13: on/off in or out
3. **Analog inputs** A0 to A5: read 0 to 5 V
4. **Power pins**: 5V, 3.3V, GND, Vin
5. **ATmega328P**: the brain, runs your sketch
6. **Reset** button: restart the sketch
7. **DC jack**: 7 to 12 V supply
8. **ON** light: the board has power

Pins you will meet



Before you power anything

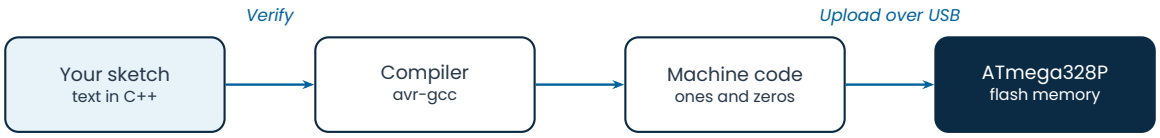
1. Check polarity: red wire to 5V, black wire to GND, never swapped.
2. Check each module's supply voltage in its datasheet before wiring it.
3. With the multimeter in continuity mode, confirm there is no short between 5V and GND.

1.2 The Arduino IDE

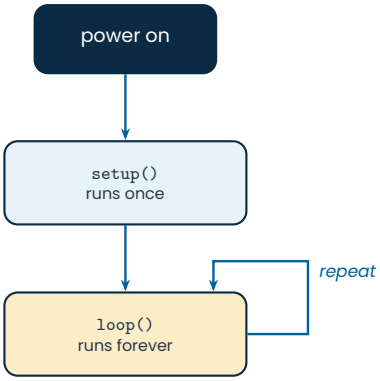


1. **Board and port**: tell the IDE which Arduino, and on which USB port
2. **Verify**: compile only, find mistakes
3. **Upload**: compile, then send to the board
4. **Serial Monitor**: text coming back from the board
5. **Output**: errors and upload progress

From text to a running chip



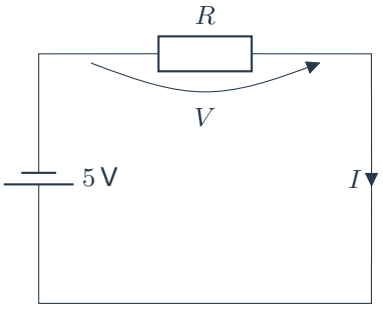
Every sketch has two parts

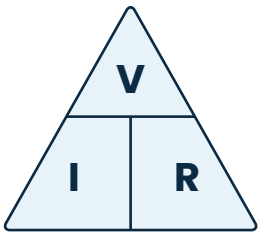


The words you will use

`pinMode`	make a pin an input or output
`digitalWrite`	set an output HIGH (5 V) or LOW (0 V)
`digitalRead`	read an input: HIGH or LOW
`analogRead`	read a voltage as a number 0 to 1023
`analogWrite`	output PWM on a ~ pin, 0 to 255
`delay`	wait, in milliseconds
`millis`	milliseconds since power on
`Serial.println`	print a line to the Serial Monitor

1.3 Electricity in 60 seconds





cover the one you want

$$V = I \times R$$
$$I = V / R$$
$$R = V / I$$

3 V over 330 Ω
→ **9 mA**
LED current: 5 V minus the LED's 2 V

5 V over 10 kΩ
→ **0.5 mA**
pull-up current (stage 3)

5 V over 1 kΩ
→ **5 mA**
transistor base (final build)

Reading a resistor



100 Ω
10 × 10

330 Ω
33 × 10

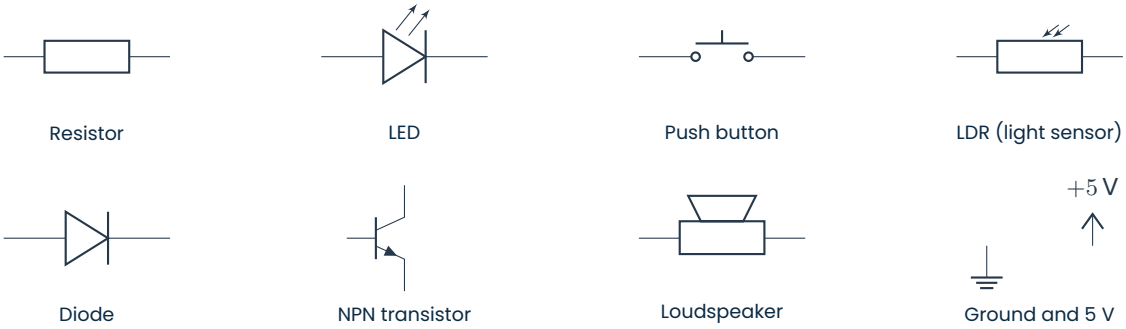
1 kΩ
10 × 100

10 kΩ
10 × 1000

COLOUR DIGITS | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |

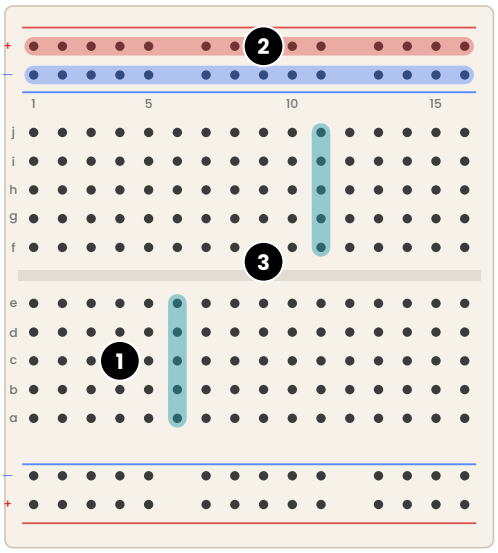
value = (band 1)(band 2) × $10^{\text{band 3}}$ · gold band = ±5%

Symbols you will meet

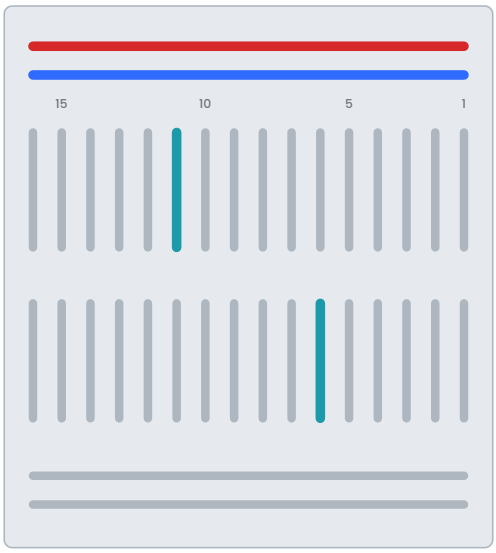


European style resistor box. Some books draw a zigzag instead; same part, same value.

1.4 The breadboard



FRONT VIEW



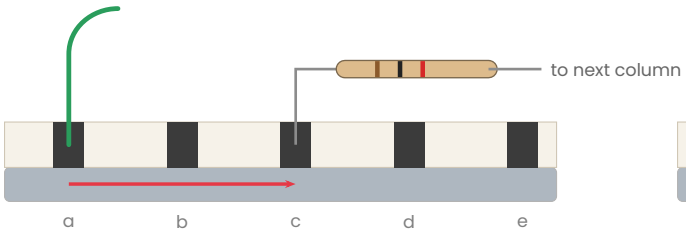
BACK VIEW (flipped left to right)

1 **Columns**: the 5 holes a to e (or f to j) of one number are one wire.

2 **Rails**: the long lines run the whole length. Use them for 5V and GND.

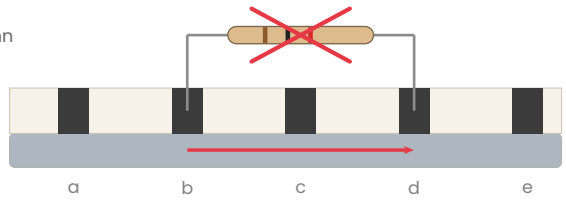
3 **Centre gap**: the two halves are not connected. Chips and buttons straddle it.

Inside one column



current runs along the strip

Right: the wire and the leg share a column, so they are one wire.



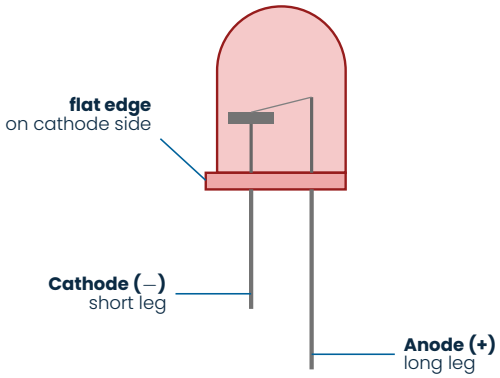
current skips the resistor

Wrong: both legs in one column, so the strip short-circuits the part.

2 Blink an LED

Your first output: one pin, one resistor, one light

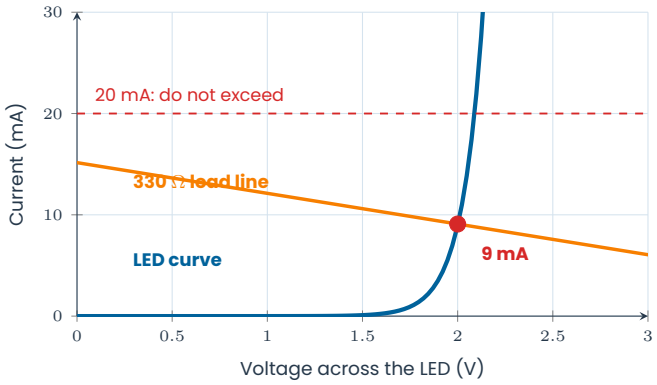
2.1 The LED





A diode is a one-way valve.

Current goes from **A**node to **K**athode, never back.



Above about 1.7 V the LED current explodes. The resistor's load line picks the working point.

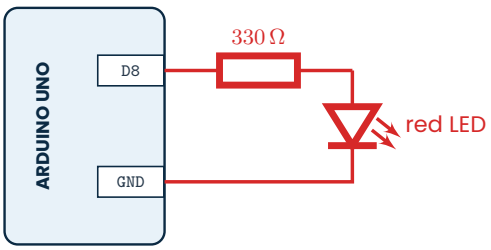
SIZING THE RESISTOR

5 V from the pin
2 V dropped by the LED
⇒ 3 V left for the resistor

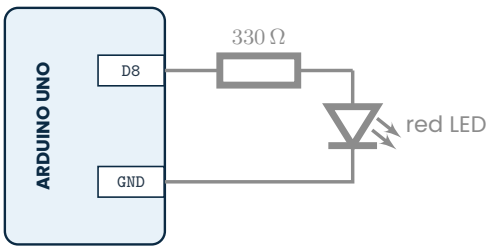
$$R = \frac{5-2}{0.009} \approx 330\,\Omega$$

A pin can give 40 mA at most, so 9 mA is safe and bright enough.

2.2 The schematic, with pin numbers

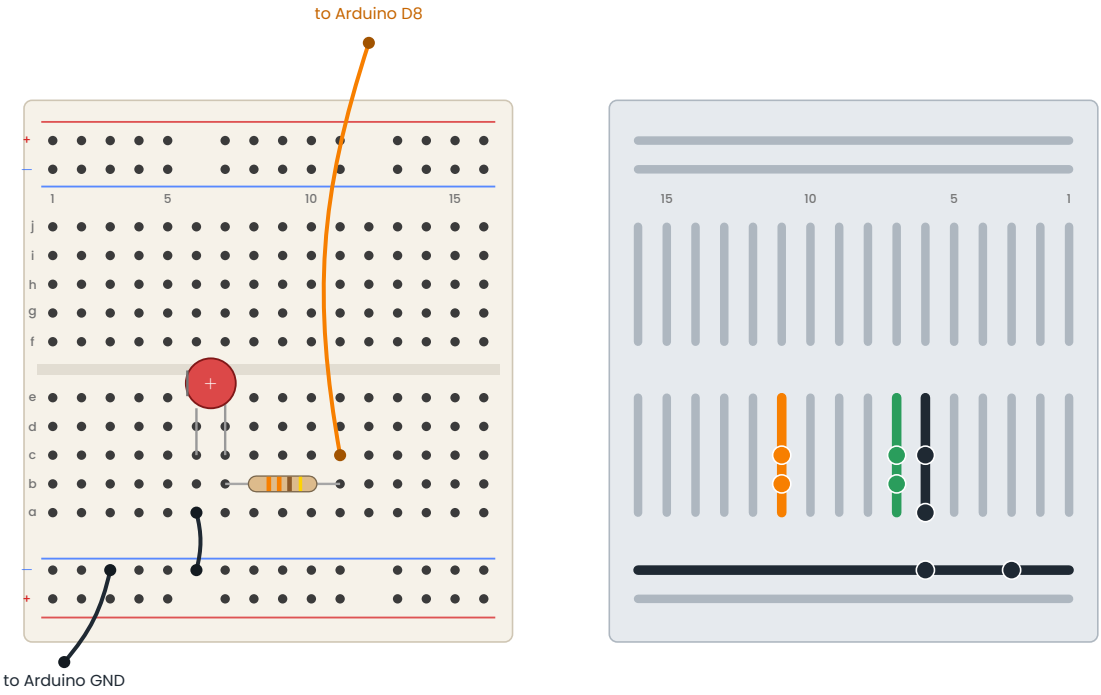


pin 8 = HIGH (5 V): current flows, LED on



pin 8 = LOW (0 V): no current, LED off

2.3 On the breadboard



- **Pin 8 net**: the jumper and one resistor leg share column 11.
- **Middle net**: the other resistor leg and the LED anode share column 7.
- **GND net**: the LED cathode shares column 6 with a jumper to the – rail.

2.4 The code

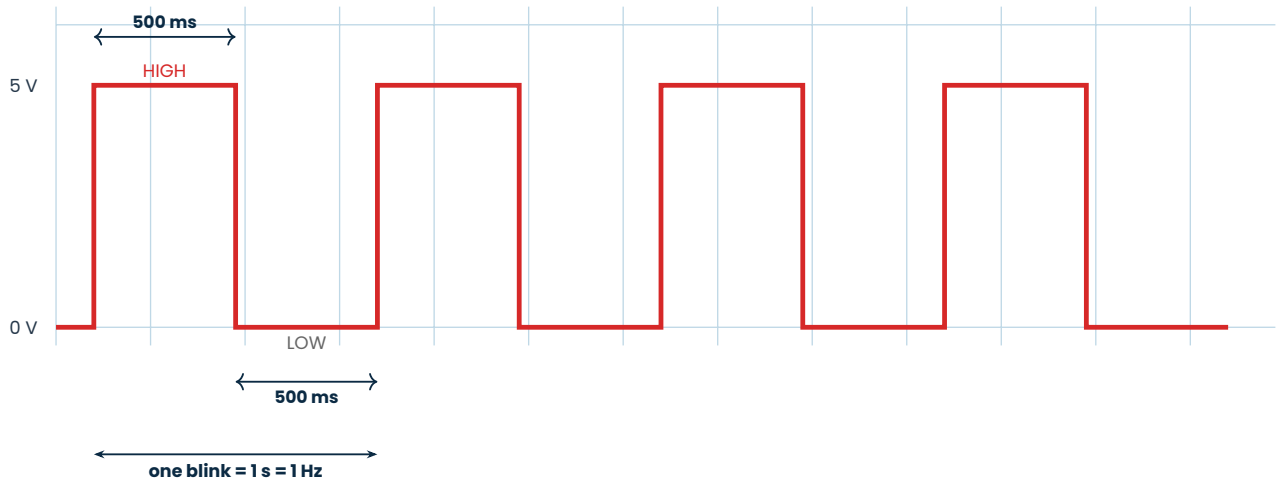
Blink.ino

```
const int LED_PIN = 8; // the LED lives on pin 8

void setup() {
  pinMode(LED_PIN, OUTPUT); // pin 8 drives things
}

void loop() {
  digitalWrite(LED_PIN, HIGH); // 5 V: LED on
  delay(500); // wait half a second
  digitalWrite(LED_PIN, LOW); // 0 V: LED off
  delay(500);
}
```

1. Give the pin a name once. Change the number in one place only.
4. Runs once at power-up. Pin 8 becomes an output.
8. `HIGH` puts 5 V on the pin.
9. The chip does nothing for 500 ms.
10. `LOW` puts 0 V on the pin.



What pin 8 does over time. The LED follows this square wave. Sound will be the same idea, only much faster.

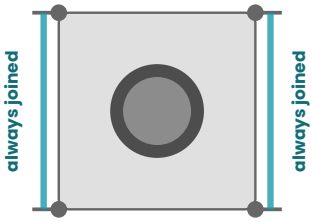
Try it

Change both `delay(500)` to `delay(100)`: faster blink. Then try `delay(5)`: the LED looks dimly on, because your eye cannot follow 100 blinks per second. Remember this, it comes back as PWM.

3 Read a button

Your first input: why a pin needs a pull-up resistor

3.1 The push button

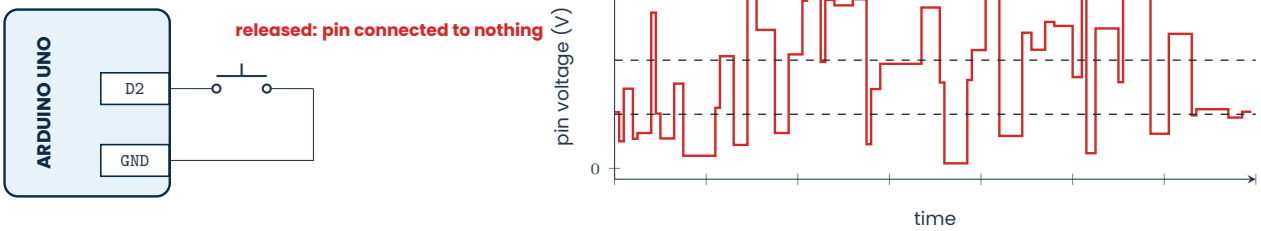


Press: left side meets right side



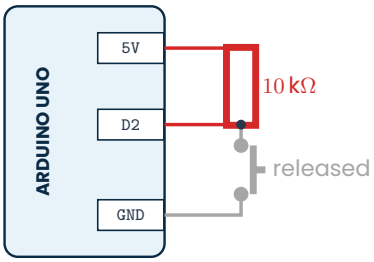
A 4-leg button is a 2-leg switch.
Legs on the same side are one wire.

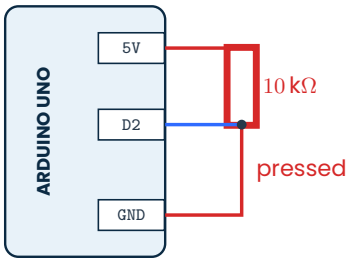
3.2 The problem: a floating pin



With nothing attached, the pin acts as an antenna. The Arduino sees random HIGH and LOW.

3.3 The fix: a pull-up resistor



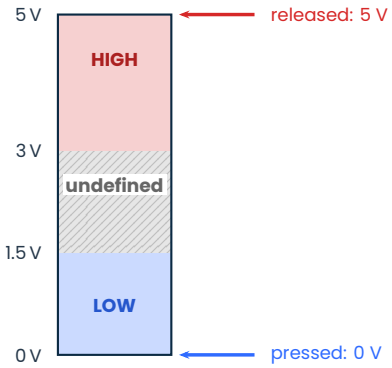


Released: nothing flows, pin = 5 V = HIGH

no current, so no voltage drops across the 10 kΩ

Pressed: 0.5 mA flows, pin = 0 V = LOW

the button wins: the pin is wired straight to GND

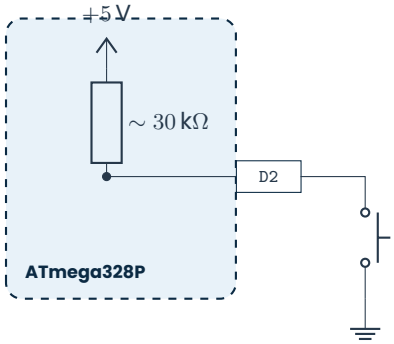


Button	Pin voltage	`digitalRead`
released	5 V	HIGH
pressed	0 V	LOW

> **Why 10 kΩ?**
> Big enough to waste only 0.5 mA when pressed.
> Small enough that the pin is firmly pulled to 5 V.

The Arduino only trusts below 1.5 V (LOW) and above 3.0 V (HIGH). Between them the reading is undefined.

3.4 The shortcut: INPUT_PULLUP



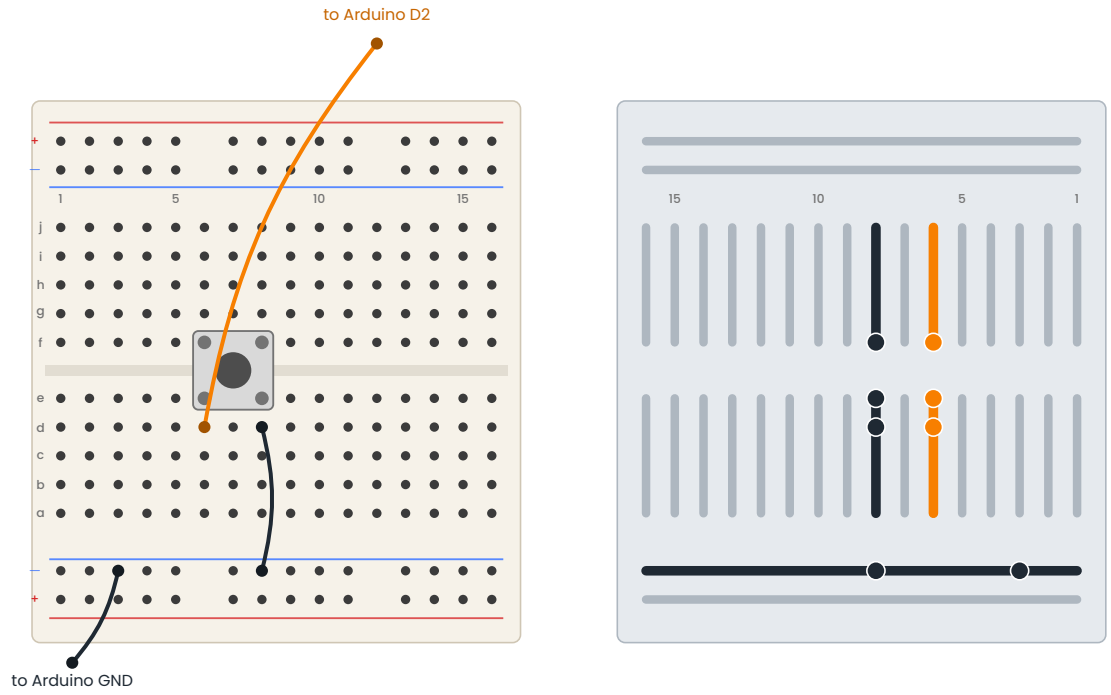
Same circuit, built in

The chip has a resistor of about 20 to 50 kΩ inside every pin. One line of code switches it on:

```
pinMode(2, INPUT_PULLUP);
```

No resistor to wire. The logic stays inverted: pressed reads LOW.

3.5 On the breadboard



FRONT VIEW **BACK VIEW**

The button straddles the centre gap. Its two left legs are joined inside, and so are its two right legs. Pressing joins left to right.

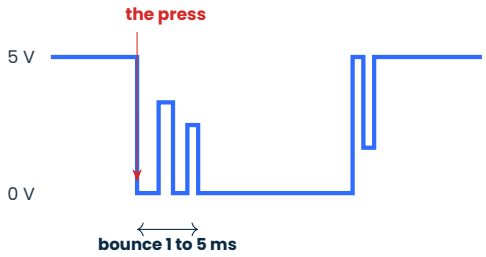
3.6 The code

ButtonLed.ino

```ino
const int BTN = 2;
const int LED = 8;

void setup() {
  pinMode(BTN, INPUT_PULLUP);
  pinMode(LED, OUTPUT);
}

void loop() {
  if (digitalRead(BTN) == LOW) { // pressed
    digitalWrite(LED, HIGH);
  } else {
    digitalWrite(LED, LOW);
  }
}
```



A real contact bounces. The chip is fast enough to see every bounce.

PressCounter.ino

```ino
int last = HIGH;
unsigned long lockUntil = 0;
int presses = 0;

void setup() {
  pinMode(2, INPUT_PULLUP);
  Serial.begin(9600);
}

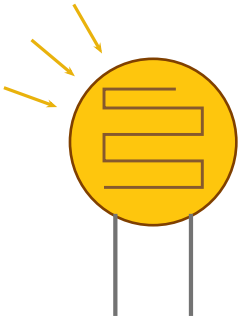
void loop() {
  int now = digitalRead(2);
  if (last == HIGH && now == LOW && millis() > lockUntil) {
    presses++; // a new press
    lockUntil = millis() + 20; // ignore bounce
    Serial.println(presses);
  }
  last = now;
}
```

1. **Edge**: count only the moment HIGH becomes LOW, not the time it stays LOW.
2. **Lockout**: after a press, ignore the pin for 20 ms so bounces are not counted.
3. **Remember**: `last` stores the previous reading for the next pass.

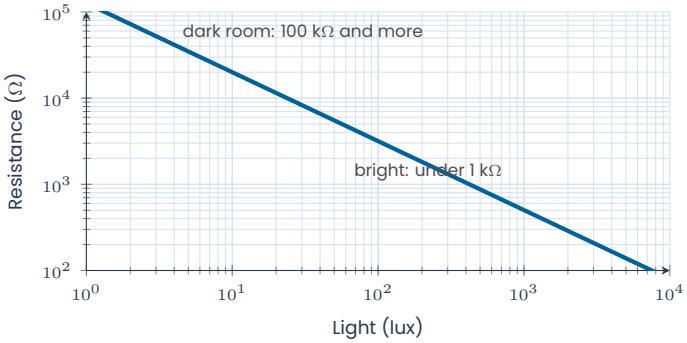
4 Read light with an LDR

Voltage dividers, the ADC, and PWM: three ideas the theremin needs

4.1 The LDR

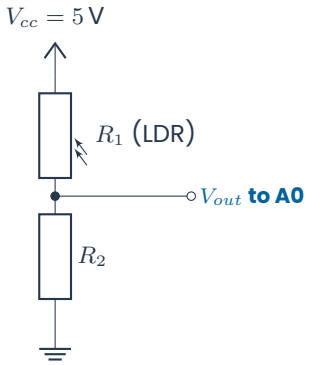


two legs, no polarity



Typical CdS cell. More light, less resistance. Your own LDR will differ.

4.2 Turning resistance into voltage

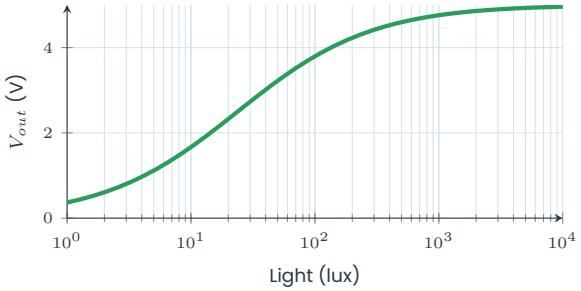


Same current through both: $I = \dfrac{V_{cc}}{R_1 + R_2}$

Voltage across R_2: $V_{out} = I \cdot R_2$

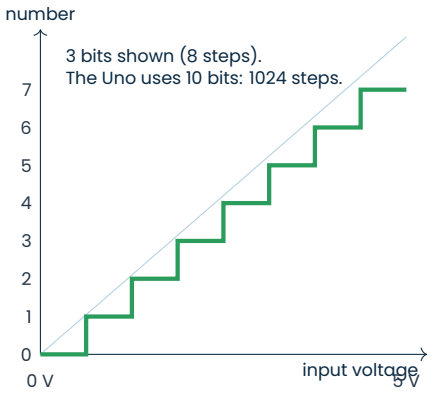
$$V_{out} = V_{cc} \frac{R_2}{R_1 + R_2}$$

Light	R_1 (LDR)	V_{out} ($R_2 = 10$ kΩ)
dark	100 kΩ	0.45 V
room	5 kΩ	3.33 V
bright	500 Ω	4.76 V



The Arduino cannot measure resistance, only voltage. The divider converts one into the other.

4.3 The ADC: voltage becomes a number



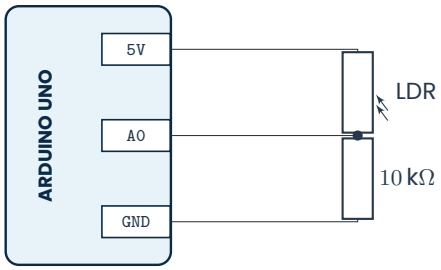
10 bits $\Rightarrow 2^{10} = 1024$ levels (0 to 1023)

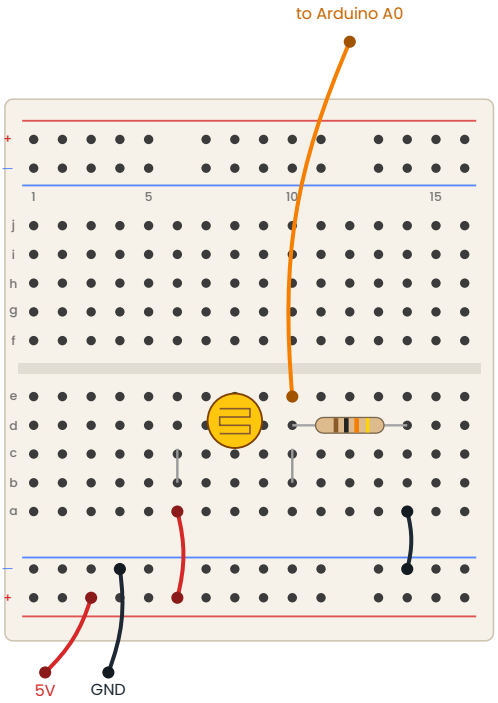
$$\text{one step} = \frac{5\,\text{V}}{1024} \approx 4.9\,\text{mV}$$

$$\text{number} = \frac{V_{in}}{5\,\text{V}} \times 1023$$

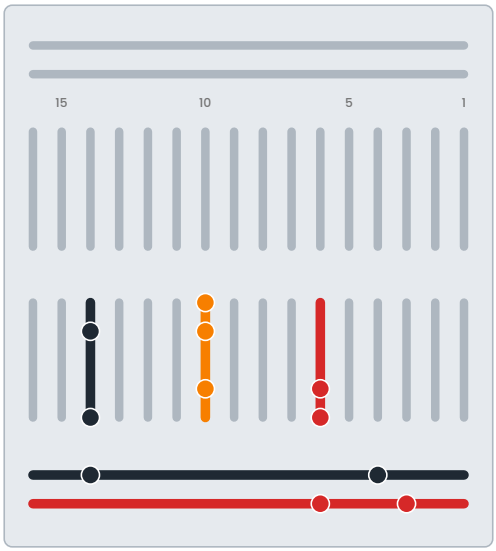
room light	3.33 V	682
hand shadow	1.0 V	205

4.4 Wiring the LDR divider





FRONT VIEW



BACK VIEW

Column 10 is the divider's middle node: LDR leg, resistor leg and the jumper to A0 all share it.

4.5 The code, and the Serial Monitor

LdrRead.ino

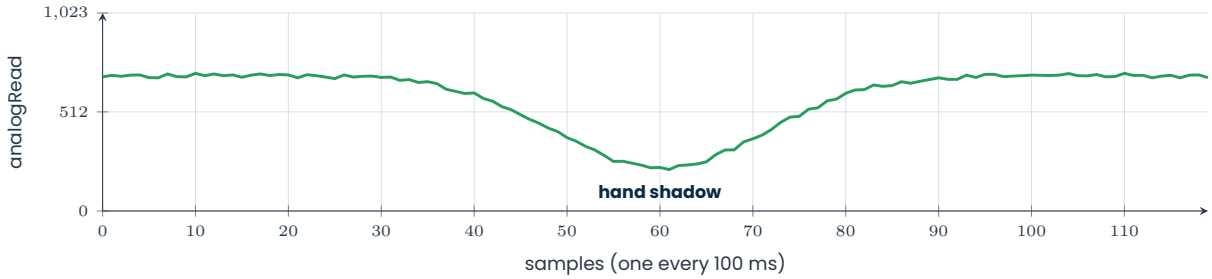
```
void setup() {
  Serial.begin(9600); // open the USB link
}

void loop() {
  int raw = analogRead(A0); // 0 to 1023
  float volts = raw * 5.0 / 1023.0; // back to volts
  Serial.print(raw);
  Serial.print(" ");
  Serial.println(volts);
  delay(100);
}
```

Serial Monitor 9600 baud

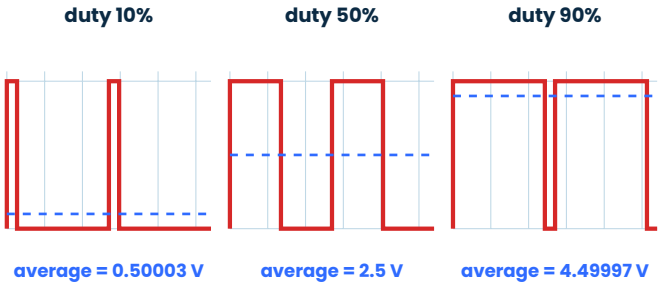
```
682  3.33
684  3.34
679  3.32
402  1.96
215  1.05
208  1.02
350  1.71
671  3.28
```

Wave your hand over the LDR: the numbers fall, then recover.



The same data on the Serial Plotter (Tools menu).

4.6 PWM: fake analog from a digital pin



The pin flips between 0 V and 5 V very fast. The LED, and later the speaker, feels only the average.

LdrDimmer.ino

```
void setup() {
  pinMode(9, OUTPUT); // pin 9 has the ~ sign
}

void loop() {
  int raw = analogRead(A0); // 0..1023
  int duty = raw / 4; // 0..255
  analogWrite(9, duty); // PWM
}
```

What you built

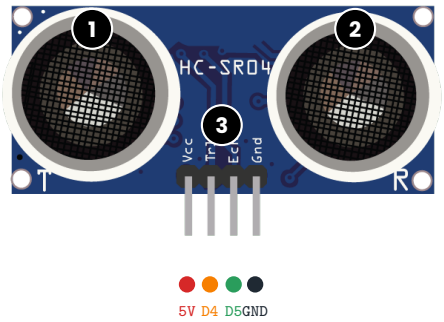
Move the LED jumper from pin 8 to pin 9. Now the LED glows brighter as the room gets brighter.

Duty cycle = fraction of time the pin is HIGH. The Uno repeats the pattern about 490 times per second on pin 9. The theremin uses the same idea for loudness.

5 Measure distance with sound

The HC-SR04 ultrasonic sensor: how it works and how we derive the code

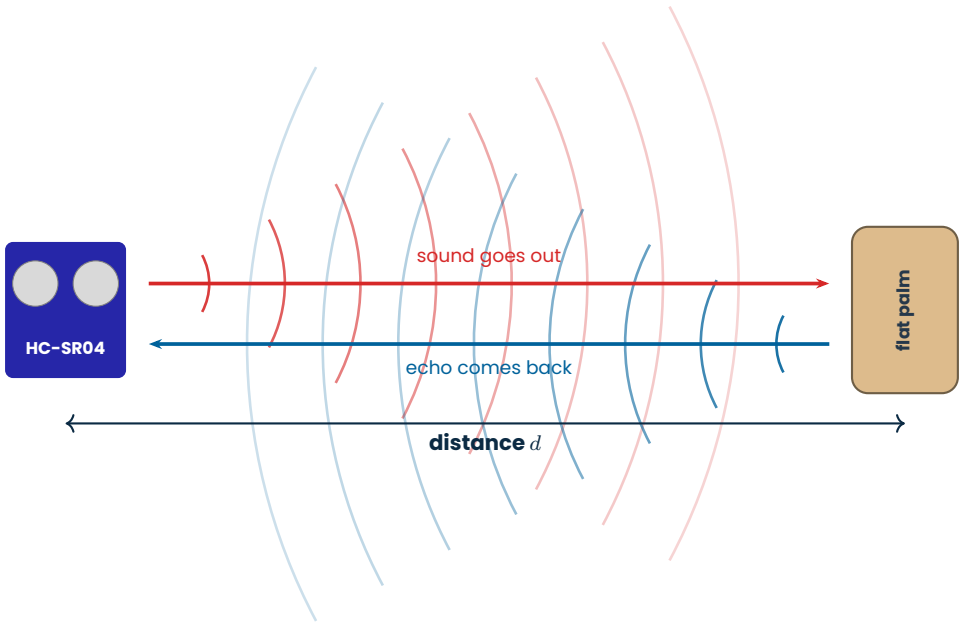
5.1 Meet the HC-SR04



1. **T** transmitter: sends a 40 kHz beep, too high to hear
2. **R** receiver: listens for the echo
3. **4 pins**: power, a start signal, and the echo answer

`Vcc`	5V	power
`Trig`	D4	Arduino says "go"
`Echo`	D5	sensor answers
`Gnd`	GND	ground

5.2 The idea: time the echo



We cannot measure d directly. We measure the time t the sound spends travelling, and compute d.

5.3 Deriving the formula

1 How fast is sound?

$$v \approx 331.3 + 0.606\,T \ \text{m/s} \quad \text{with } T \text{ in °C} \quad \Rightarrow \quad v(20\,^\circ\text{C}) \approx 343 \ \text{m/s}$$

2 There and back

The echo crosses the distance twice: $2d = v \cdot t \quad \Rightarrow \quad d = \dfrac{v\,t}{2}$

3 Choose friendly units

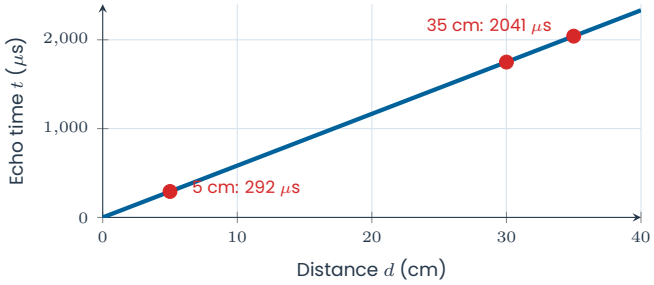
$$343 \ \text{m/s} = 34\,300 \ \text{cm/s} = 0.0343 \ \text{cm}/\mu\text{s}$$

$$d_{\text{cm}} = \frac{0.0343 \cdot t_{\mu s}}{2} = \boxed{0.01715 \cdot t_{\mu s}}$$

4 Read it backwards

$$\frac{1}{0.01715} \approx 58 \quad \Rightarrow \quad \text{every } \mathbf{58\ \mu s} \text{ of echo time is } \mathbf{1\ cm}$$

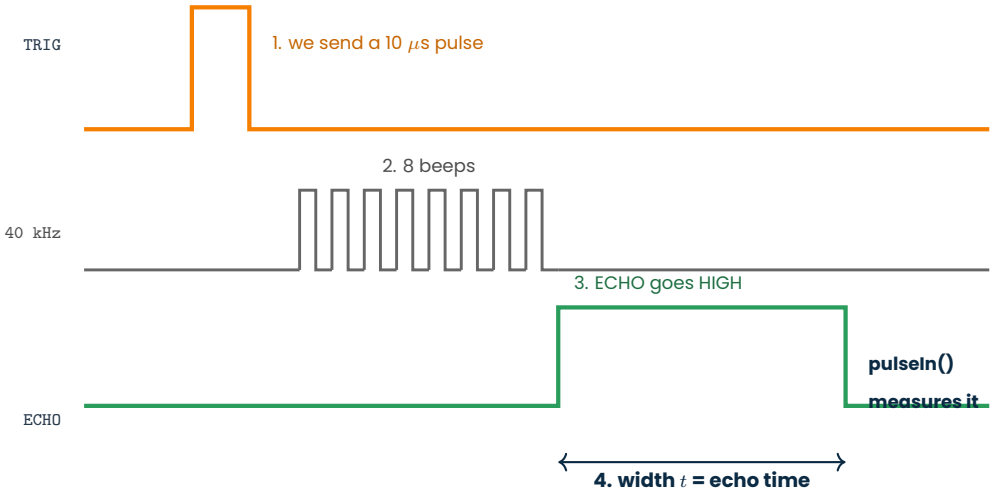
Check: $t = 1750\ \mu s \ \Rightarrow \ d = 1750 \times 0.01715 = 30.0$ cm



Air temp.	v (m/s)	error at 30 cm
0 °C	331	−1.0 cm
20 °C	343	0
35 °C	353	+0.9 cm

Temperature changes the speed of sound a little. For a hand 5 to 35 cm away we ignore it.

5.4 What the sensor does, step by step



Not to scale. The 8 beeps last only 200 µs. The echo pulse lasts up to a few thousand µs.

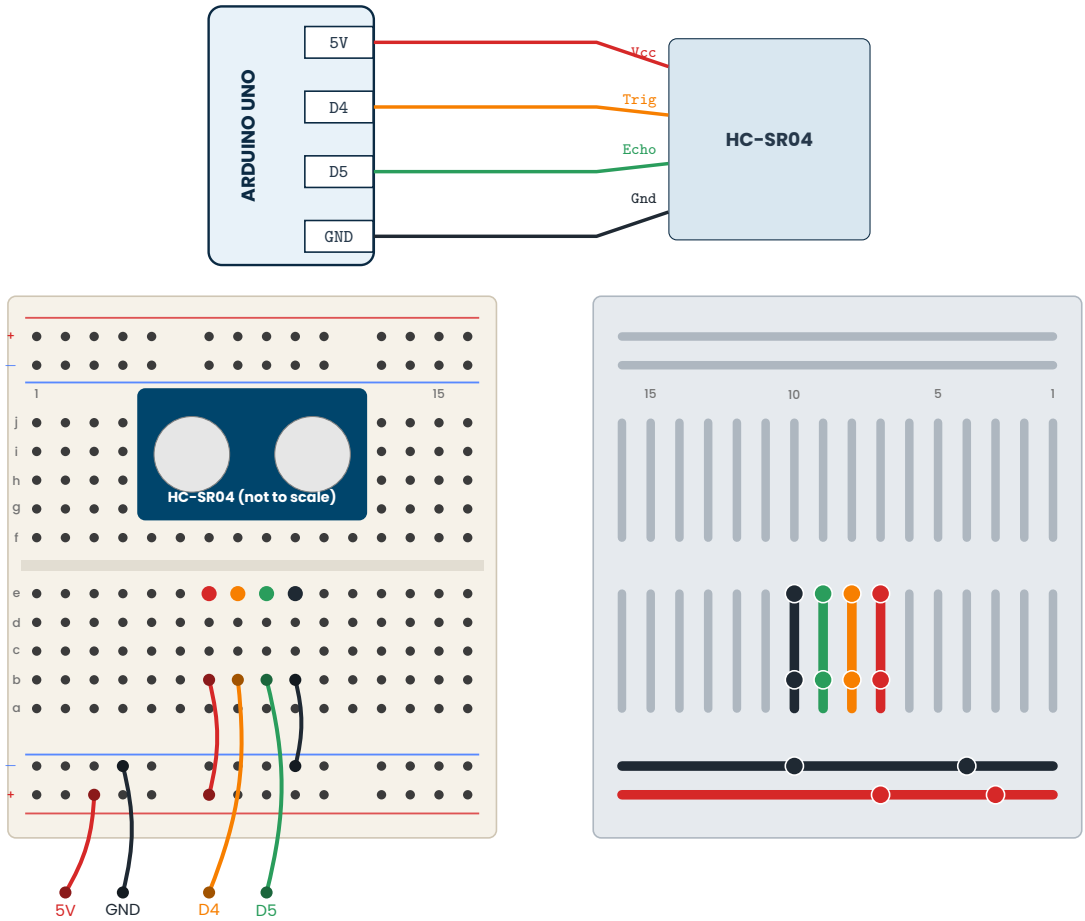
5.5 The code, line by line

Distance.ino

```
1  const int TRIG = 4;
2  const int ECHO = 5;
3
4  void setup() {
5    pinMode(TRIG, OUTPUT);
6    pinMode(ECHO, INPUT);
7    Serial.begin(9600);
8  }
9
10 float readCm() {
11   digitalWrite(TRIG, LOW);
12   delayMicroseconds(2); // clean start
13   digitalWrite(TRIG, HIGH);
14   delayMicroseconds(10); // the 10 us pulse
15   digitalWrite(TRIG, LOW);
16   long t = pulseIn(ECHO, HIGH, 3000); // width in us
17   if (t == 0) return -1; // no echo
18   return t * 0.01715; // d = v t / 2
19 }
20
21 void loop() {
22   Serial.println(readCm());
23   delay(60);
24 }
```

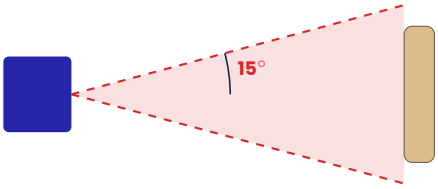
1. Two pins, two jobs: Arduino talks on D4, listens on D5.
2. A 10 µs HIGH on TRIG starts one measurement.
3. `pulseIn` times how long ECHO stays HIGH, in µs.
4. The 3000 µs limit gives up after about 51 cm, so an empty air space never freezes the program.
5. Step 3 of the derivation, in one line.
6. 60 ms between pings lets old echoes die out.

5.6 Wiring the sensor



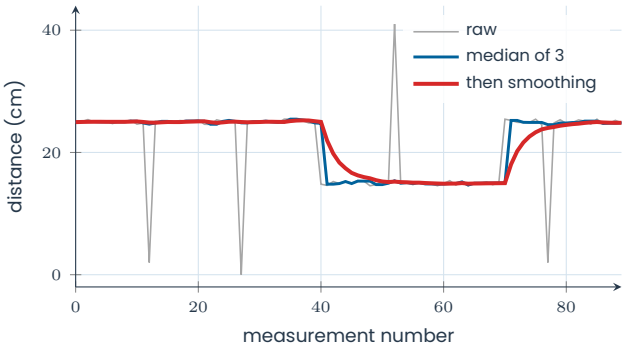
The module plugs into columns 7 to 10. Each pin gets its own column, so each pin gets its own wire.

5.7 Limits, and why we filter



Beam is a 15° cone. Aim it at a flat palm, keep the other hand and the body out of it.

Range 2 to 400 cm, **resolution** about 3 mm.



A hand moves from 25 cm to 15 cm and back. Four bad readings (0, 2, 41 cm) appear in the raw data.

FILTER 1: MEDIAN OF 3

keep the last 3 readings, sort, take the middle one

kills single spikes

FILTER 2: SMOOTHING

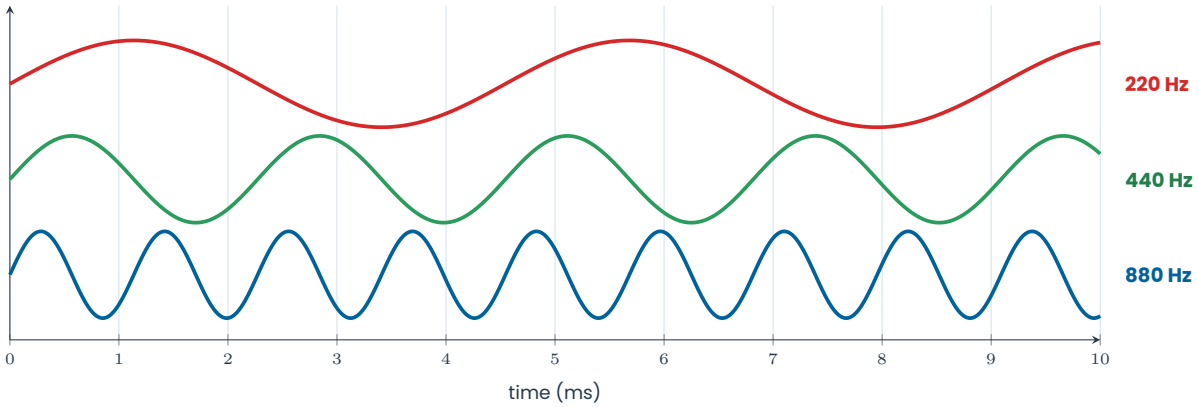
$$s_{\text{new}} = 0.7\, s_{\text{old}} + 0.3\, x$$

blends each reading into a running value

6 Pitch, tone and volume

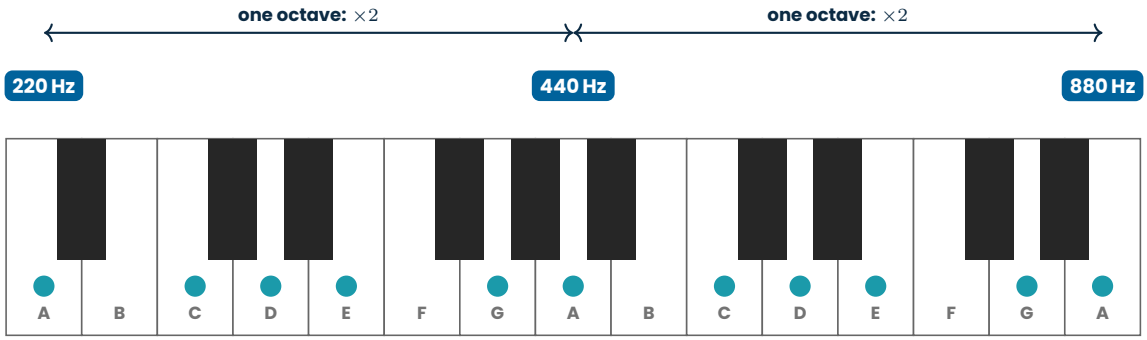
What a note is, and how the Arduino makes one with timers

6.1 Pitch is how fast the air shakes



Frequency = shakes per second, in hertz (Hz). Twice the frequency sounds exactly one octave higher.

6.2 Notes are not evenly spaced



● = the A minor pentatonic scale (A, C, D, E, G): any order sounds musical

12 semitones per octave

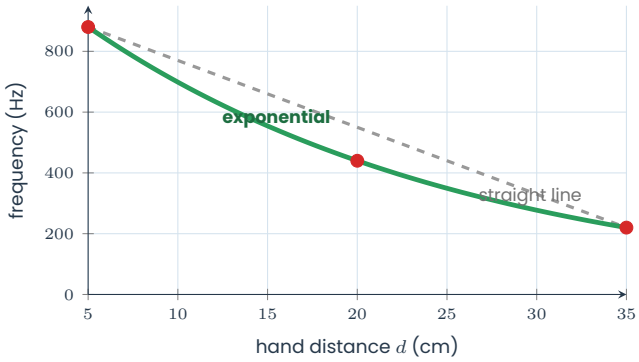
$$\frac{f_{\text{next}}}{f_{\text{now}}} = 2^{1/12} \approx 1.0595$$

each step up is about 6% faster

Note	Frequency	Step from the one below
A3	220 Hz	
A4	440 Hz	+220 Hz
A5	880 Hz	+440 Hz

Equal musical steps are equal ratios, not equal differences. The next octave needs twice as many hertz.

6.3 From hand distance to frequency



Hand at	Note	f
5 cm	A5	880 Hz
20 cm	A4	440 Hz
35 cm	A3	220 Hz

A straight line would put A4 at 550 Hz, an out-of-tune note. Only the exponential curve hits every octave exactly.

1 We want two octaves in 30 cm

$$d = 5 \text{ cm} \rightarrow 880 \text{ Hz} \qquad d = 35 \text{ cm} \rightarrow 220 \text{ Hz} \qquad \text{ratio} = \frac{220}{880} = \frac{1}{4}$$

2 Each cm multiplies the pitch by the same factor

$$f(d) = 880 \cdot \left(\frac{220}{880}\right)^{\frac{d-5}{30}} = 880 \cdot 4^{-\frac{d-5}{30}} = 880 \cdot 2^{-\frac{d-5}{15}}$$

3 How wide is one note?

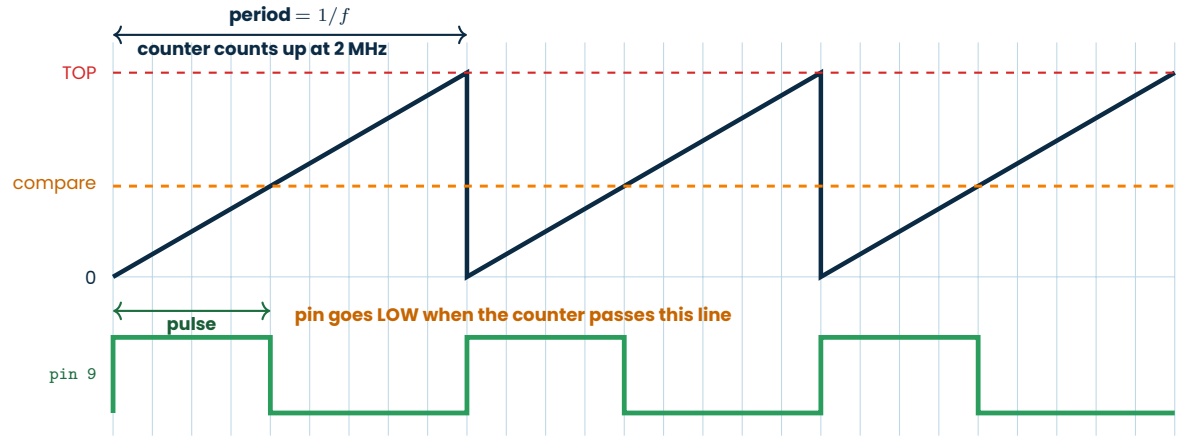
$$2 \text{ octaves} = 24 \text{ semitones in } 30 \text{ cm} \Rightarrow \frac{30 \text{ cm}}{24} = \boxed{1.25 \text{ cm per semitone}}$$

The sensor resolves about 3 mm, so a semitone spans only 4 steps. That is why the readings are filtered.

6.4 Making the wave with a timer

Why not tone()?

The built-in `tone()` uses Timer2 and always makes a 50% square wave. Frequency is easy, but loudness is fixed. We need to control the pulse width too, so we program Timer1 directly.



A counter counts from 0 to TOP and restarts. TOP sets the frequency. The compare line sets how long the pin stays HIGH. The hardware does this alone, no code runs.

FREQUENCY

$$f = \frac{F_{CPU}}{8\,(\text{TOP}+1)} \quad \text{with } F_{CPU} = 16\,\text{MHz}$$

$$\boxed{\text{TOP} = \frac{16\,000\,000}{8\,f} - 1}$$

PULSE WIDTH

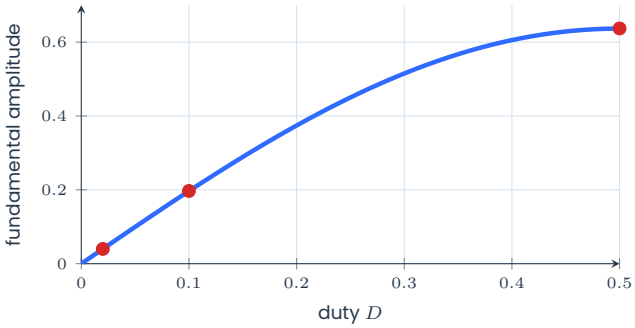
$$\text{compare} = \text{TOP} \times D \quad (D = \text{duty, 0 to 1})$$

Note	f	TOP
A5	880 Hz	2271
A4	440 Hz	4544
A3	220 Hz	9089

The 16-bit Timer1 counts up to 65 535, so it reaches down to about 30 Hz. One count at 880 Hz is a 0.04% error, far below the 6% of a semitone.

6.5 Volume from pulse width





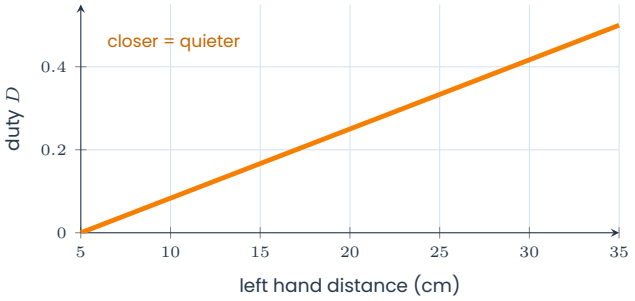
$$\text{amplitude} = \frac{2}{\pi}\sin(\pi D)$$

$$D = 0.50 \Rightarrow 0.64$$
$$D = 0.10 \Rightarrow 0.20$$
$$D = 0.02 \Rightarrow 0.04$$

0.64 against 0.04 is a range of 16 to 1

Trade-off

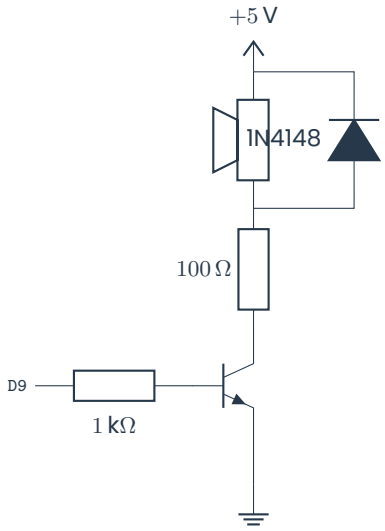
Narrow pulses are quieter but also thinner and brighter in tone, because they carry more high harmonics. That is the price of volume control with a single pin.



$$D = \text{clamp}\left(0.5 \cdot \frac{d_L - 5}{30},\ 0.02,\ 0.5\right)$$

as on a real theremin: hand near the antenna means silence

6.6 The output stage: a transistor drives the speaker



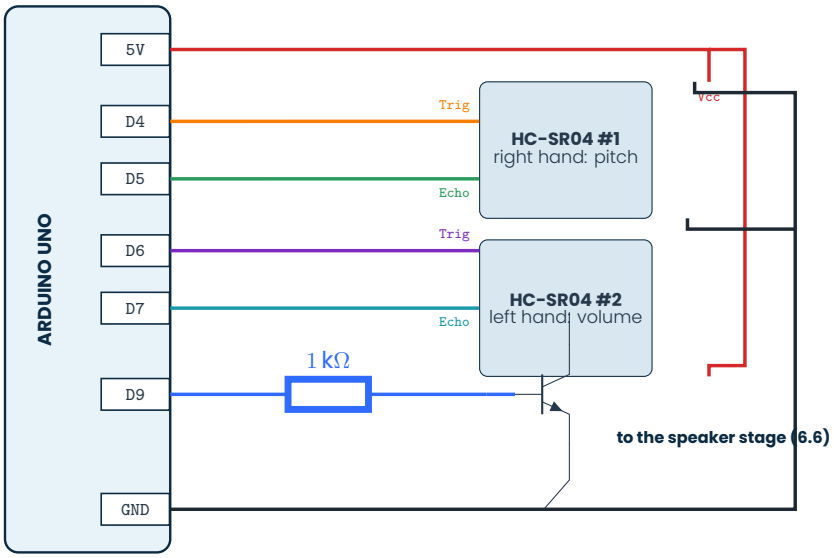
1. A pin cannot drive a speaker: 5 V over 8 Ω is 625 mA, the pin allows 40 mA.
2. The transistor is a switch. 4.3 mA into its base switches about 44 mA through the speaker.
3. 100 Ω limits the speaker current.
4. The diode catches the voltage spike when the coil turns off.

Pin 9 sends the PWM wave to the transistor's base. The speaker sits between +5 V and the collector.

7 Put it all together

The full circuit, how the program thinks, and how to debug it

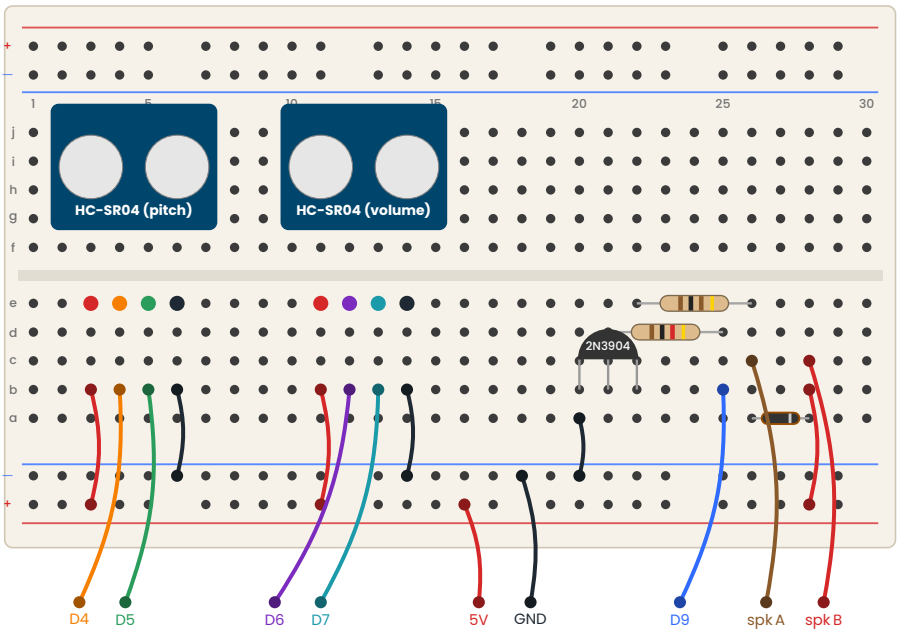
7.1 The complete wiring



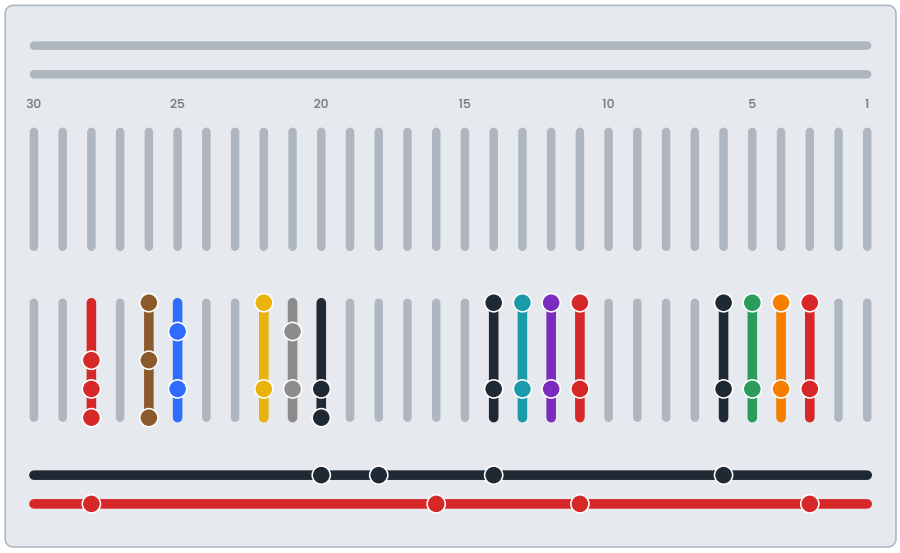
Block wiring. 5V and GND feed both sensors. The transistor stage is drawn in detail in section 6.6.

Pitch sensor	`D4, D5`	Trig and Echo of the right-hand sensor
Volume sensor	`D6, D7`	Trig and Echo of the left-hand sensor
Audio out	`D9`	Timer1 output OC1A, to the transistor base through 1 kΩ
Mode switch	`D2`	optional: continuous or quantised, to GND with INPUT_PULLUP

7.2 The complete breadboard



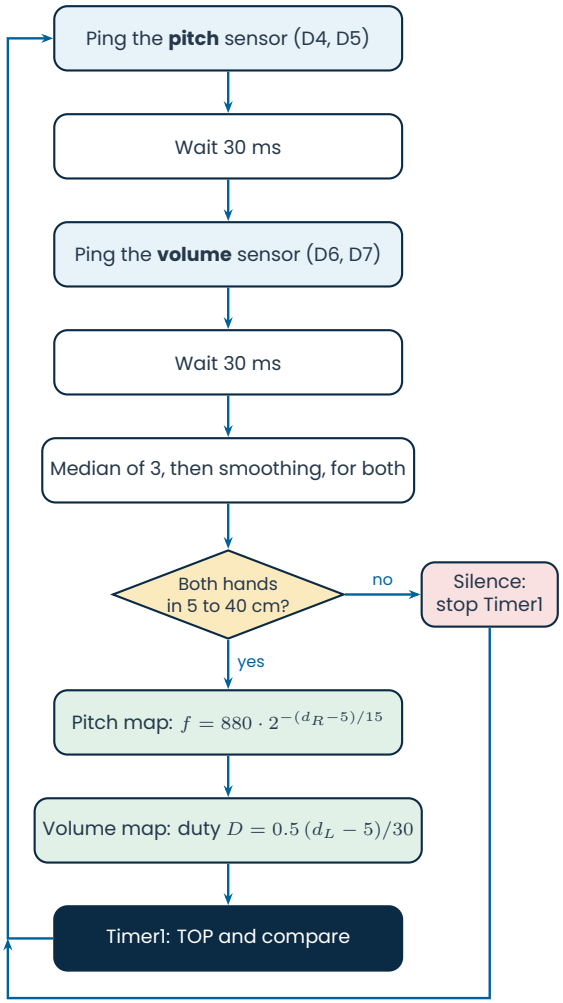
FRONT VIEW



BACK VIEW

Left: sensor 1 (pitch) in columns 3 to 6, sensor 2 (volume) in columns 11 to 14. Right: the transistor stage. Colours match between the two views.

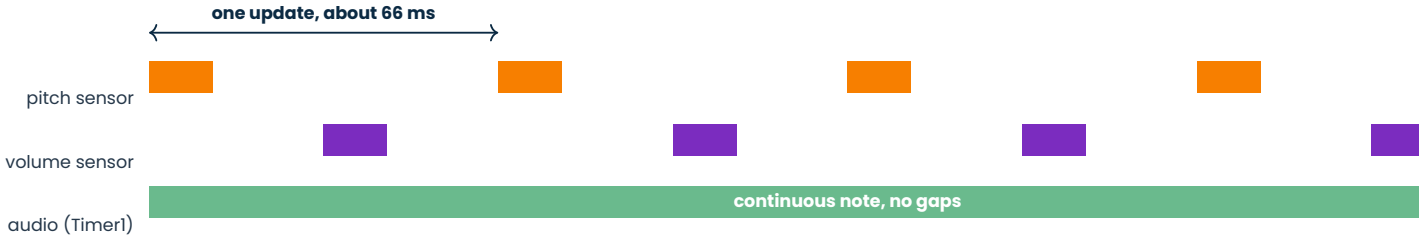
7.3 How the program thinks



While the loop runs

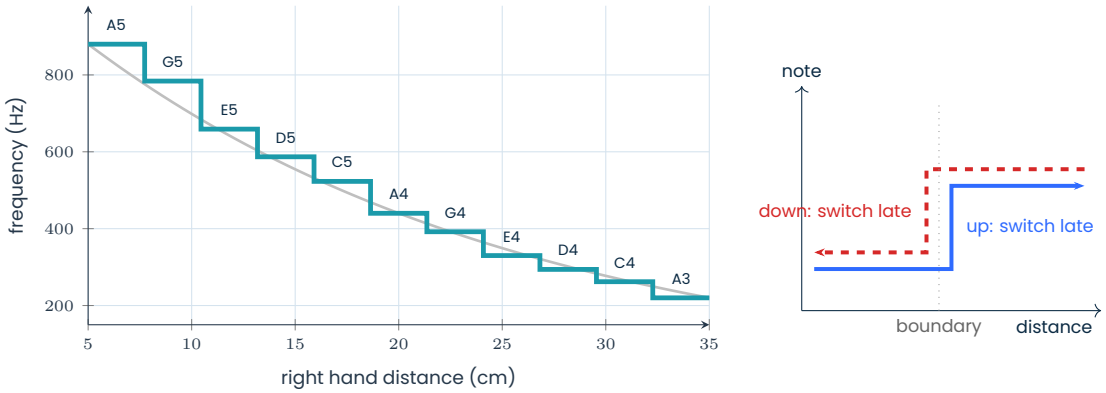
Timer1 keeps making the wave in hardware. The code only changes TOP and the compare value, about 15 times per second. The note never stops while the sensors wait for echoes.

Alternate	Never ping both sensors at once: each hears the other's 40 kHz burst.
30 ms	Gives each echo time to die out first.
Tubes	Cardboard tubes around each sensor narrow the beam.



Sensors take turns. The sound never waits for them.

7.4 Quantised mode: notes snap to a scale



Left: 11 pentatonic notes laid over the 30 cm. Right: hysteresis. The note only changes after the hand passes 0.3 of a note width beyond the boundary, so it does not warble.

7.5 When it does not work

Symptom	Likely cause	Check
No sound at all	Transistor backwards, or D9 not reaching the 1 kΩ	Pin order E, B, C on your datasheet; follow column 25 on the back view
Constant tone	Volume sensor seeing nothing, or duty stuck high	Print d_L on the Serial Monitor
Pitch jumps around	Spikes and a body in the beam	Tubes on the sensors, flat palm, check the median filter
One hand changes both	Sensors hearing each other	Point them away from each other, keep 30 ms between pings
Board resets when loud	Speaker current sags the 5 V rail	Power from a wall adapter, not only USB
Hum when no hand	Timer1 left running	Make sure the silence branch stops Timer1

Before you power it

1. Transistor flat face toward you, pins read E, B, C (check your part).
2. Diode stripe toward +5 V.
3. Multimeter on continuity: no beep between 5V and GND.

7.6 Challenges

Vibrato
Wobble the frequency slowly with a low-frequency oscillator

LED bar
Light one LED for each note you play

Two scales
Use the D2 switch to pick pentatonic or major